



Set Up Kubernetes Deployment

CH

Arpita Chowdhury

```
ec2-user@ip-172-31-25-198 nextwork-flask-backend$ docker build -t nextwork-flask-backend .
[*] Building ll.1s (10/10) FINISHED
--> [internal] load build definition from Dockerfile
--> transferring dockerfile: 249B
--> [internal] load metadata for docker.io/library/python:3.9-alpine
--> [internal] load .dockerignore
--> transferring context: 2B
--> [1/5] FROM docker.io/library/python:3.9-alpine#sha256:c99b6eb43b3ac4d75db3d6e8b22268d5ea9a99deead7218c6d5da7f2ca029c
--> resolve docker.io/library/python:3.9-alpine#sha256:c99b6eb43b3ac4d75db3d6e8b22268d5ea9a99deead7218c6d5da7f2ca029c
--> sha256:c99b6eb43b3ac4d75db3d6e8b22268d5ea9a99deead7218c6d5da7f2ca029c 10.29KB / 10.29KB
--> sha256:ca181c0e08421bd0c2a0d2e6d1890f91267f250a1f261807a882286e15d7708 1.78KB / 1.78KB
--> sha256:ac3f0e0ec2244c78e0c8b273d6ad02f8437f0ca4a1cf0f75727f502114470d 5.18KB / 5.18KB
--> sha256:2d35ebdb57d9971fe0cac1582aa78935adf855b2cc32db163c9882265d4a1b 3.80MB / 3.80MB
--> sha256:c900817b6dd8d904c7a87e38fa6c8b0e04f7d87df8e9061284b12345949451 456.92KB / 456.92KB
--> sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf864ffa2933d7c3db866d 15.40MB / 15.40MB
--> extracting sha256:2d35ebdb57d9971fe0cac1582aa78935adf855b2cc32db163c9882265d4a1b
--> sha256:980924f60da4a3d740c42410d70aa25164f0c8a8498131c0e974c0eacabeb31 247B / 247B
--> extracting sha256:c900817b6dd8d904c7a87e38fa6c8b0e04f7d87df8e9061284b12345949451
--> extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf864ffa2933d7c3db866d
--> extracting sha256:980924f60da4a3d740c42410d70aa25164f0c8a8498131c0e974c0eacabeb31
--> [internal] load build context
--> transferring context: 42.30KB
--> [2/5] WORKDIR /app
--> [3/5] COPY requirements.txt requirements.txt
--> [4/5] RUN pip install -r requirements.txt
--> [5/5] COPY . .
--> exporting to image
--> exporting layers
--> writing image sha256:118add9e555dab591e1cd037a2f11cf0d2d0a02e075950994f55c09b4115e
--> naming to docker.io/library/nextwork-flask-backend
ec2-user@ip-172-31-25-198 nextwork-flask-backend$
```



Introducing Today's Project!

In this project, I will clone a real backend application from GitHub, containerize it using Docker, and push the resulting image to an Amazon ECR repository so it can be used in cloud-native deployments. I will also troubleshoot build, configuration, and authentication issues that commonly occur when working with Docker and cloud container registries.

Tools and concepts

I used Amazon EKS, Git, Docker, and Amazon ECR to build, package, and prepare a real backend application for deployment in a cloud-native Kubernetes environment. Git allowed me to pull the application code, Docker was used to containerize the backend, ECR stored the container image, and EKS provided the scalable platform where the application can run. Key steps include cloning the backend repository from GitHub, building a Docker image using a Dockerfile, fixing permissions and configuration issues during the build process, pushing the image to Amazon ECR, and preparing an EKS cluster with managed node groups to run the containerized application.

Project reflection

This project took me approximately 2 hours to complete, including setting up the EKS environment, building and containerizing the backend, and pushing the image to Amazon ECR. The most challenging part was troubleshooting the Docker permissions and configuration issues while building the container image, especially understanding how Linux user permissions and the Docker daemon interact in a cloud environment. My favourite part was seeing the backend successfully packaged into a Docker image and pushed to Amazon ECR, knowing that it was now ready to be deployed and scaled in a real Kubernetes cluster.

Something new that I learned from this experience was how container images



What I'm deploying

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included running this command: `eksctl create cluster \ --name nextwork-eks-cluster \ --nodegroup-name nextwork-nodegroup \ --node-type t3.micro \ --nodes 3 \ --nodes-min 1 \ --nodes-max 3 \ --version 1.33 \ --region us-east-1` in my Ec2 instance terminal I created for this project.

I'm deploying an app's backend

Next, I retrieved the backend that I plan to deploy. An app's backend is the server-side part of the application that handles requests, runs business logic, and communicates with data sources. It is responsible for processing user actions and returning the correct responses to the frontend. I retrieved the backend code by using Git to clone the `nextwork-flask-backend` repository from GitHub onto my EC2 instance. This copied all of the application files, including the Flask app, Dockerfile, and dependency list, into a local project folder so I could build, containerize, and deploy the backend to my EKS cluster.



```
[ec2-user@ip-172-31-25-198 ~]$ git clone https://github.com/NatNextWork1/nextwork-flask-backend.git
Cloning into 'nextwork-flask-backend'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 18 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), 6.14 KiB | 2.05 MiB/s, done.
Resolving deltas: 100% (4/4), done.
[ec2-user@ip-172-31-25-198 ~]$ ls
nextwork-flask-backend
[ec2-user@ip-172-31-25-198 ~]$
```




Building a container image

Once I cloned the backend code, my next step is to build a container image of the backend. This is because a container image packages the application and all of its dependencies into a single, portable unit that can run the same way on any system. By building the image using the Dockerfile, I ensure the backend is configured exactly as intended by the development team, making it easy to deploy consistently across development, testing, and production environments such as Amazon EKS.

When I tried to build a Docker image of the backend, I ran into a permissions error because the ec2-user did not have permission to access the Docker daemon socket (/var/run/docker.sock). Docker runs as a root-owned service, and only users who are part of the docker group (or the root user) are allowed to communicate with it. Since my user was not yet added to the Docker group, Docker blocked the build command for security reasons.

To solve the permissions error, I added my ec2-user to the Docker group using the usermod command. This allowed my user to communicate with the Docker daemon without needing to use sudo for every Docker command. The Docker group is a Linux user group that grants permission to interact with the Docker engine. Users in this group are allowed to access the Docker socket and manage containers, images, and builds, making it possible to run Docker commands securely and efficiently without full root access.



```
[ec2-user@ip-172-31-25-198 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
[+] Building 11.1s (10/10) FINISHED                                docker:default
-> [internal] load build definition from Dockerfile               0.0s
-> => transferring dockerfile: 269B                               0.0s
-> [internal] load metadata for docker.io/library/python:3.9-alpine 0.4s
-> [internal] load .dockerignore                                  0.0s
-> => transferring context: 2B                                     0.0s
-> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb41b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3dada7f2ca029c 1.7s
-> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb41b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3dada7f2ca029c 0.0s
-> => sha256:c99b6eb41b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3dada7f2ca029c 10.29kB / 10.29kB 0.0s
-> => sha256:a381ce066821bdce2a0d62e601890f912e9f23d9e1fb61007e58328863bd7708 1.73kB / 1.73kB 0.0s
-> => sha256:ac5fec9ec244c78ecb822308adfb0f849f6c0aa1ef08f75572f6d52124470d 5.18kB / 5.18kB 0.0s
-> => sha256:2d15e6db57d9971feadce015f2a079935edfd8058b2cc125b145c9892e6d41b 3.80kB / 3.80kB 0.1s
-> => sha256:c9008197b8dd9d9b4c7a87e38fa6c8bceb4cf7d87df8e9061284b123459f9451 456.92kB / 456.92kB 0.1s
-> => sha256:27711589a506c37675206ab7f5d51377b4e63d501c1866ff9a2933d7c3db896d 15.40kB / 15.40kB 0.4s
-> => extracting sha256:2d15e6db57d9971feadce015f2a079935edfd8058b2cc125b145c9892e6d41b 0.3s
-> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecab3b3 247B / 247B 0.2s
-> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa6c8bceb4cf7d87df8e9061284b123459f9451 0.1s
-> => extracting sha256:27711589a506c37675206ab7f5d51377b4e63d501c1866ff9a2933d7c3db896d 0.4s
-> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecab3b3 0.0s
-> [internal] load build context                                0.0s
-> => transferring context: 42.30kB                               0.0s
-> [2/5] WORKDIR /app                                          0.1s
-> [3/5] COPY requirements.txt requirements.txt                0.1s
-> [4/5] RUN pip3 install -r requirements.txt                  7.5s
-> [5/5] COPY . .                                              0.1s
-> exporting to image                                          1.0s
-> => exporting layers                                           1.0s
-> => writing image sha256:118add99e55dab591ec1dc037a7211fc0d2d0a002e075950994f55c09b4115e 0.0s
-> => naming to docker.io/library/nextwork-flask-backend 0.0s
[ec2-user@ip-172-31-25-198 nextwork-flask-backend]$
```



Container Registry

I'm using Amazon ECR in this project to store and manage the Docker image of my backend application so it can be securely pulled and deployed by my EKS cluster. ECR is a good choice for the job because it is fully integrated with AWS services like EKS and IAM, which makes authentication, access control, and image pulling seamless and secure. It also supports image scanning for vulnerabilities and provides highly available, scalable storage, making it ideal for production-grade containerized applications.

Container registries like Amazon ECR are great for Kubernetes deployment because they provide a secure, reliable, and centralized place to store container images that Kubernetes can pull from whenever it needs to start or scale applications. Instead of each node building or storing its own copy of an application, Kubernetes simply downloads the exact same verified image from the registry, ensuring consistency across all pods and environments. ECR also integrates directly with Amazon EKS and IAM, which makes authentication and image access automatic and secure. This allows Kubernetes to quickly launch new containers during scaling or recovery events, making deployments faster, more reliable, and production-ready.



nextwork-flask-backend

Summary

Images

Lifecycle policy

Permissions

Repository tags

Images (1) [Info](#)

[Delete](#) [Copy URI](#) [Details](#) [Scan](#) [View push commands](#)

☐	Image tags	Type	Created at	Image size	Image digest	Last pulled at
☐	latest	Image	January 11, 2026, 16:36:39 (UTC-06)	39.99	sha256:413250686...	-



EXTRA: Backend Explained

After reviewing the app's backend code, I've learned that the application is a lightweight Flask-based REST API that pulls live data from an external service and exposes it through a simple HTTP endpoint. It uses flask-restx to structure the API, requests to call the Hacker News API, and Flask to handle incoming client requests. The backend is designed to be stateless and easy to containerize, making it ideal for running in Docker and Kubernetes environments where services need to scale and restart reliably.

Unpacking three key backend files

The requirements.txt file lists all of the Python libraries and dependencies that the Flask backend needs in order to run correctly, such as Flask itself and any other packages the application relies on. When the Dockerfile runs `pip3 install -r requirements.txt`, it installs these exact dependencies inside the container, ensuring that the application has everything it needs to start and behave the same way in every environment where the image is deployed.

placeholder

The Dockerfile gives Docker instructions on how to build a container image for the backend application, including which base operating system and programming language to use, which dependencies to install, where to place the application files, and how to start the app when the container runs. Key commands in this Dockerfile include `FROM`, which defines the base image (Python on Alpine Linux); `WORKDIR`, which sets the working directory inside the container; `COPY`, which brings the application code and dependency files into the image; `RUN`, which installs the required Python packages; and `CMD`, which tells the container how to start the Flask application when it is launched.

The app.py file contains the main backend logic for the application. It defines a Flask-based REST API that listens for requests and returns data to clients.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

